

**TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN**  
**KHOA TOÁN KINH TẾ**

**BÀI TẬP LỚN**

**Học phần: Học sâu (Deep Learning)**

**ĐỀ TÀI: PHÂN LOẠI TRÁI CÂY TƯƠI VÀ HỎNG**  
**BẰNG MẠNG NƠ-RON TÍCH CHẬP**

**Giảng viên hướng dẫn:** TS. Nguyễn Thị Kim Ngân

**Sinh viên thực hiện:**

Triệu Hải Đăng Trình - [MSSV]

Trương Hoàng Tùng - [MSSV]

[Họ tên Quân] - [MSSV]

Nguyễn Dương Hiếu - [MSSV]

**Hà Nội, tháng 10 năm 2026**

# MỤC LỤC

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| PHẦN 1. CƠ SỞ LÝ THUYẾT VỀ MẠNG NƠ-RON TÍCH CHẬP (CNN) .....               | 3  |
| 1.1. Từ mạng nơ-ron truyền thẳng đến mạng nơ-ron tích chập .....           | 3  |
| 1.2. Lớp tích chập (convolutional layer).....                              | 4  |
| 1.3. Lớp gộp (pooling layer) .....                                         | 5  |
| 1.4. Lớp kết nối đầy đủ và lớp đầu ra .....                                | 5  |
| 1.5. Huấn luyện CNN.....                                                   | 6  |
| 1.6. Các kiến trúc CNN tiêu biểu .....                                     | 8  |
| 1.7. Học chuyển giao (transfer learning) và tinh chỉnh (fine-tuning) ..... | 9  |
| 1.8. Tăng cường dữ liệu (data augmentation) .....                          | 10 |
| 1.9. Học đa nhiệm (multi-task learning) .....                              | 10 |
| 1.10. Đánh giá mô hình phân loại .....                                     | 11 |
| 1.11. Giải thích mô hình bằng Grad-CAM .....                               | 11 |
| PHẦN 2. ỨNG DỤNG THỰC TIỄN .....                                           | 13 |
| I. Mô tả bài toán .....                                                    | 13 |
| II. Mô tả bộ dữ liệu .....                                                 | 14 |
| III. Thiết kế mô hình CNN.....                                             | 16 |
| 1. Tiền xử lý dữ liệu: làm sạch, chuẩn hoá và tăng cường .....             | 16 |
| 2. Mô hình 1 - CNN tuần tự đơn giản .....                                  | 20 |
| 3. Mô hình 2 - CNN phức tạp đa nhiệm (Functional API) .....                | 21 |
| 4. Mô hình 3 - Học chuyển giao và tinh chỉnh MobileNetV2 .....             | 22 |
| 5. Quy trình huấn luyện chung.....                                         | 23 |
| TÀI LIỆU THAM KHẢO.....                                                    | 24 |

# PHẦN 1. CƠ SỞ LÝ THUYẾT VỀ MẠNG NƠ-RON TÍCH CHẬP (CNN)

## 1.1. Từ mạng nơ-ron truyền thẳng đến mạng nơ-ron tích chập

**Nơ-ron nhân tạo.** Một nơ-ron nhận vector đầu vào  $\mathbf{x}$ , tính tổng có trọng số rồi đưa qua hàm kích hoạt  $f$ :

$$z = \mathbf{w}^T \mathbf{x} + b, \quad a = f(z).$$

Các hàm kích hoạt thường dùng là sigmoid  $\sigma(z) = \frac{1}{1+e^{-z}}$ , tanh và ReLU  $f(z) = \max(0, z)$ . Sigmoid có đạo hàm  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$  tiến về 0 khi  $|z|$  lớn nên dễ gây hiện tượng triệt tiêu gradient (vanishing gradient); ReLU có đạo hàm bằng 1 với  $z > 0$ , tính toán nhanh và là lựa chọn mặc định cho các lớp ẩn của CNN hiện đại.

**Mạng nhiều lớp (MLP).** Các nơ-ron được xếp thành lớp; với lớp thứ  $l$ :

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}), \quad \mathbf{a}^{(0)} = \mathbf{x}.$$

Toàn bộ trọng số được học bằng cách tối thiểu hoá hàm mất mát  $J$  với thuật toán gradient descent, trong đó gradient của  $J$  theo từng  $\mathbf{W}^{(l)}, \mathbf{b}^{(l)}$  được tính bằng lan truyền ngược (backpropagation) dựa trên quy tắc chuỗi:

$$\begin{aligned} \delta^{(L)} &= \frac{\partial J}{\partial \mathbf{a}^{(L)}} \odot f'(\mathbf{z}^{(L)}), \quad \delta^{(l)} = (\mathbf{W}^{(l+1)})^T \delta^{(l+1)} \odot f'(\mathbf{z}^{(l)}), \quad \frac{\partial J}{\partial \mathbf{W}^{(l)}} \\ &= \delta^{(l)} (\mathbf{a}^{(l-1)})^T. \end{aligned}$$

**Hạn chế của MLP với dữ liệu ảnh.** Một ảnh màu  $224 \times 224 \times 3$  có 150.528 giá trị; chỉ một lớp ẩn 1.000 nơ-ron đã cần khoảng 150 triệu trọng số. MLP còn bỏ qua cấu trúc không gian của ảnh (các điểm ảnh gần nhau có liên quan mật thiết) và phải học lại cùng một mẫu (ví dụ một vết thâm trên vỏ quả) ở mọi vị trí. Mạng nơ-ron tích chập khắc phục điều này bằng ba ý tưởng: **kết nối cục bộ** (mỗi nơ-ron chỉ nhìn một vùng nhỏ), **chia sẻ trọng số** (cùng một bộ lọc được trượt trên toàn ảnh) và **gộp (pooling)** để giảm kích thước. Một CNN điển hình gồm: Ảnh đầu vào  $\rightarrow$  [Lớp tích chập + Lớp gộp]  $\times n \rightarrow$  Lớp kết nối đầy đủ  $\rightarrow$  Đầu ra.

## 1.2. Lớp tích chập (convolutional layer)

**Phép tương quan chéo (cross-correlation).** Lớp tích chập trượt một mảng hạt nhân (kernel)  $K$  kích thước  $k_h \times k_w$  trên mảng đầu vào  $X$ ; tại mỗi vị trí, nhân từng phần tử rồi cộng lại:

$$Y[i, j] = \sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} X[i+a, j+b] K[a, b] + b_0.$$

Ví dụ với đầu vào  $3 \times 3$  gồm các số  $0 \dots 8$  và kernel  $\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}$ : phần tử đầu tiên của đầu ra là  $0 \cdot 0 + 1 \cdot 1 + 3 \cdot 2 + 4 \cdot 3 = 19$ , và toàn bộ đầu ra là  $\begin{bmatrix} 19 & 25 \\ 37 & 43 \end{bmatrix}$ . Giá trị của kernel không được thiết kế thủ công mà được học từ dữ liệu bằng gradient descent; các kernel ở lớp đầu thường học được các bộ phát hiện cạnh, màu sắc, kết cấu, còn các lớp sâu hơn kết hợp chúng thành các mẫu phức tạp (hình dạng quả, vết thối, nấm mốc). Mảng đầu ra được gọi là bản đồ đặc trưng (feature map).

**Kích thước đầu ra, đệm (padding) và bước trượt (stride).** Với đầu vào  $n_h \times n_w$  và kernel  $k_h \times k_w$ , đầu ra có kích thước  $(n_h - k_h + 1) \times (n_w - k_w + 1)$ . Đệm thêm tổng cộng  $p_h$  hàng và  $p_w$  cột (thường là số 0) quanh viền, và trượt với bước  $s_h, s_w$ , kích thước đầu ra là

$$\left\lceil \frac{n_h - k_h + p_h + s_h}{s_h} \right\rceil \times \left\lceil \frac{n_w - k_w + p_w + s_w}{s_w} \right\rceil.$$

Chọn  $p_h = k_h - 1$ ,  $p_w = k_w - 1$  (đệm “same”) và  $s = 1$  giữ nguyên chiều cao, chiều rộng; stride 2 giảm một nửa kích thước.

**Nhiều kênh đầu vào và đầu ra.** Ảnh màu có  $c_i = 3$  kênh (R, G, B). Khi đó mỗi kernel có kích thước  $c_i \times k_h \times k_w$ ; phép tương quan chéo được thực hiện trên từng kênh rồi cộng kết quả các kênh lại. Để tạo  $c_o$  kênh đầu ra, lớp dùng  $c_o$  kernel như vậy. Số tham số của một lớp tích chập là  $(k_h k_w c_i + 1) c_o$  - **không phụ thuộc kích thước ảnh**, nhờ chia sẻ trọng số. Ví dụ lớp Conv  $3 \times 3$  từ 3 lên 32 kênh có  $(3 \cdot 3 \cdot 3 + 1) \cdot 32 = 896$  tham số (lớp đầu tiên của Mô hình 1).

**Tích chập  $1 \times 1$ .** Kernel  $1 \times 1$  không nhìn lân cận không gian mà chỉ kết hợp các kênh tại từng điểm ảnh - tương đương một lớp kết nối đầy đủ áp dụng độc lập cho

mỗi vị trí. Nó được dùng để thay đổi số kênh và thêm tính phi tuyến (NiN, GoogLeNet, ResNet, MobileNetV2).

**Vùng tiếp nhận (receptive field).** Mỗi phân tử ở lớp sâu phụ thuộc vào một vùng ngày càng lớn của ảnh gốc khi các lớp tích chập và gộp được xếp chồng; nhờ vậy mạng học được đặc trưng từ cục bộ (cạnh) đến toàn cục (cả quả).

### 1.3. Lớp gộp (pooling layer)

Lớp gộp thường đặt giữa các lớp tích chập, có hai tác dụng: (1) giảm độ nhạy của biểu diễn đối với vị trí chính xác của đặc trưng và (2) giảm kích thước biểu diễn, do đó giảm chi phí tính toán của các lớp sau. Cửa sổ gộp  $p \times q$  trượt trên đầu vào như kernel nhưng **không có tham số**:

- **Gộp cực đại (max pooling)** lấy giá trị lớn nhất trong cửa sổ - với đầu vào  $0 \dots 8$  và cửa sổ  $2 \times 2$  ta được  $\max(0,1,3,4) = 4$ ,  $\max(1,2,4,5) = 5$ ,  $\max(3,4,6,7) = 7$ ,  $\max(4,5,7,8) = 8$ ;
- **Gộp trung bình (average pooling)** lấy giá trị trung bình trong cửa sổ.

Đệm và bước trượt áp dụng như lớp tích chập; phổ biến nhất là cửa sổ  $2 \times 2$ , stride 2, giảm một nửa chiều cao và chiều rộng ( $224 \times 224 \times 64 \rightarrow 112 \times 112 \times 64$ ). Với đầu vào nhiều kênh, lớp gộp xử lý **từng kênh riêng biệt**, nên số kênh đầu ra bằng số kênh đầu vào.

**Gộp trung bình toàn cục (global average pooling - GAP)** lấy trung bình toàn bộ bản đồ đặc trưng của mỗi kênh, biến tensor  $H \times W \times D$  thành vector  $D$  chiều. GAP được NiN đề xuất và dùng trong GoogLeNet, ResNet, MobileNetV2 vì giảm mạnh số tham số so với Flatten + Dense và ít bị quá khớp hơn.

### 1.4. Lớp kết nối đầy đủ và lớp đầu ra

Sau các khối tích chập - gộp, tensor đầu ra  $H \times W \times D$  được duỗi phẳng (flatten) thành vector  $H \cdot W \cdot D$  chiều (hoặc gộp bằng GAP), rồi đi qua các lớp kết nối đầy đủ (fully connected, Dense) - thực chất là một MLP - để kết hợp các đặc trưng và tính đầu ra.

**Phân loại nhiều lớp - softmax và cross-entropy.** Với  $C$  lớp, lớp đầu ra tính  $\mathbf{z} = \mathbf{W}^T \mathbf{x} + \mathbf{b}$  rồi chuẩn hoá thành phân phối xác suất bằng hàm softmax:

$$a_i = \frac{\exp(z_i)}{\sum_{j=1}^C \exp(z_j)}, \quad i = 1, \dots, C.$$

Hàm mất mát là cross-entropy giữa nhãn one-hot  $\mathbf{y}$  và dự đoán  $\mathbf{a}$ ; với nhãn đúng  $y$  của mẫu thì

$$J = - \sum_{j=1}^C y_j \log a_j = -\log a_y, \quad \frac{\partial J}{\partial \mathbf{z}} = \mathbf{a} - \mathbf{y}.$$

Trong Keras, `SparseCategoricalCrossentropy` là cùng hàm mất mát này nhưng nhận nhãn dạng số nguyên thay vì one-hot.

**Phân loại nhị phân - sigmoid và binary cross-entropy.** Với hai lớp (ví dụ tươi / hỏng), chỉ cần một nơ-ron đầu ra  $\hat{y} = \sigma(z) = P(y = 1 | \mathbf{x})$  và hàm mất mát

$$J = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})],$$

chính là hàm log-likelihood âm của hồi quy logistic. Dự đoán nhãn 1 khi  $\hat{y} \geq 0,5$ .

## 1.5. Huấn luyện CNN

**Gradient descent và các biến thể.** Tham số được cập nhật ngược hướng gradient:  $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w})$  với tốc độ học (learning rate)  $\eta$ . Batch gradient descent dùng toàn bộ tập huấn luyện cho mỗi bước (chính xác nhưng chậm); stochastic gradient descent dùng một mẫu (rẻ nhưng nhiễu); **mini-batch gradient descent** dùng một nhóm nhỏ  $B$  mẫu (ví dụ 32) - cân bằng giữa độ ổn định và tốc độ, tận dụng tính toán ma trận, và là cách huấn luyện mặc định:

$$\nabla J_B(\mathbf{w}) = \frac{1}{|B|} \sum_{(x,y) \in B} \nabla J_{xy}(\mathbf{w}).$$

Tốc độ học quá nhỏ làm hội tụ chậm; quá lớn làm hàm mất mát dao động hoặc phân kỳ.

**Adam** (Kingma & Ba, 2015) kết hợp quán tính (momentum) và chuẩn hoá theo độ lớn gradient (RMSProp), tự điều chỉnh tốc độ học cho từng tham số:

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, & \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, & \mathbf{w}_t \\ &= \mathbf{w}_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}, \end{aligned}$$

với  $\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t)$ ,  $\hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t)$  (thường  $\beta_1 = 0,9$ ,  $\beta_2 = 0,999$ ).

**Triệt tiêu gradient và cách khắc phục.** Trong mạng sâu  $o = f_L \circ f_{L-1} \circ \dots \circ f_1(\mathbf{x})$ , gradient theo các lớp đầu là tích của nhiều ma trận đạo hàm; nếu các thừa số nhỏ, gradient gần như bằng 0 và các lớp đầu không học được. Các giải pháp: hàm kích hoạt ReLU / Leaky ReLU; khởi tạo trọng số phù hợp - Xavier/Glorot  $W \sim U\left[-\frac{\sqrt{6}}{\sqrt{n_{in}+n_{out}}}, \frac{\sqrt{6}}{\sqrt{n_{in}+n_{out}}}\right]$  giữ phương sai tín hiệu ổn định qua các lớp; chuẩn hoá theo lô (batch normalization) và kết nối tắt (residual connection).

**Chuẩn hoá theo lô (batch normalization - BN)** (Ioffe & Szegedy, 2015). Giá trị kích hoạt ở các lớp trung gian có thể biến thiên rất khác nhau giữa các lớp và theo thời gian huấn luyện, gây khó hội tụ. BN chuẩn hoá mỗi đặc trưng theo thống kê của mini-batch  $\mathcal{B}$  rồi khôi phục bậc tự do bằng hai tham số học được  $\gamma$  (scale) và  $\beta$  (shift):

$$\hat{\mu}_B = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} \mathbf{x}, \quad \hat{\sigma}_B^2 = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} (\mathbf{x} - \hat{\mu}_B)^2 + \epsilon, \quad \text{BN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \hat{\mu}_B}{\hat{\sigma}_B} + \beta.$$

Khi suy luận, BN dùng trung bình và phương sai trượt (moving average) tích lũy trong quá trình huấn luyện. Với lớp tích chập, thống kê được tính theo từng kênh trên toàn bộ batch và mọi vị trí không gian.

**Chống quá khớp (overfitting).**

- *Chính quy hoá L2 (ridge / weight decay)*: thêm  $\frac{\lambda}{2} \|\mathbf{W}\|^2$  vào hàm mất mát để phạt trọng số lớn.
- *Dropout* (Srivastava et al., 2014): ở mỗi bước huấn luyện, ngẫu nhiên đặt về 0 một tỉ lệ  $p$  (thường  $\leq 0,5$ ) nơ-ron của một lớp, buộc mạng không phụ thuộc vào một vài nơ-ron cụ thể; khi suy luận dropout bị tắt.
- *Dừng sớm (early stopping)*: theo dõi hàm mất mát trên tập kiểm định và dừng khi nó không còn giảm, giữ lại trọng số tốt nhất.
- *Giảm tốc độ học khi chững lại (ReduceLROnPlateau)*: nhân tốc độ học với một hệ số nhỏ hơn 1 khi hàm mất mát kiểm định không cải thiện sau vài epoch.
- *Tăng cường dữ liệu* (mục 1.8).

## 1.6. Các kiến trúc CNN tiêu biểu

**LeNet-5** (LeCun et al., 1998) - CNN đầu tiên được ứng dụng thực tế (nhận dạng chữ số viết tay): hai lớp tích chập  $5 \times 5$  (6 và 16 kênh, kích hoạt sigmoid), mỗi lớp theo sau bởi gộp trung bình  $2 \times 2$  stride 2, rồi ba lớp kết nối đầy đủ 120 - 84 - 10.

**AlexNet** (Krizhevsky et al., 2012) - sâu hơn nhiều (5 lớp tích chập, 2 lớp ẩn kết nối đầy đủ và 1 lớp đầu ra), dùng ReLU thay sigmoid, dropout cho các lớp kết nối đầy đủ, ảnh đầu vào  $224 \times 224$ ; chiến thắng ImageNet 2012 và mở ra kỷ nguyên học sâu cho thị giác máy tính.

**VGG** (Simonyan & Zisserman, 2014) - xây dựng mạng từ các **khối (block)** lặp lại: mỗi khối VGG gồm vài lớp tích chập  $3 \times 3$  (padding 1) và một lớp gộp cực đại  $2 \times 2$  stride 2 (giảm một nửa độ phân giải). VGG-11 có 5 khối (8 lớp tích chập, số kênh tăng gấp đôi từ 64 đến 512) và 3 lớp kết nối đầy đủ. Ý tưởng “mạng = chồng các khối” được dùng trong hầu hết các kiến trúc sau này.

**NiN - Network in Network** (Lin et al., 2013) - mỗi khối NiN gồm một lớp tích chập theo sau bởi hai lớp tích chập  $1 \times 1$  (đóng vai trò lớp kết nối đầy đủ tại từng điểm ảnh); lớp cuối có số kênh bằng số lớp và được gộp trung bình toàn cục thay cho các lớp kết nối đầy đủ - giảm đáng kể số tham số.

**GoogLeNet - mạng đa nhánh** (Szegedy et al., 2015) - khối Inception có bốn nhánh song song: tích chập  $1 \times 1$ ;  $1 \times 1$  rồi  $3 \times 3$ ;  $1 \times 1$  rồi  $5 \times 5$ ; gộp cực đại  $3 \times 3$  rồi  $1 \times 1$ . Các nhánh trích xuất thông tin ở nhiều kích thước không gian, được cộng để giữ nguyên chiều cao - chiều rộng và nối lại theo chiều kênh. GoogLeNet xếp 9 khối Inception thành ba nhóm, ngăn cách bởi gộp cực đại, và kết thúc bằng GAP.

**ResNet - mạng phần dư** (He et al., 2016) - trong một khối thông thường, các lớp phải học trực tiếp ánh xạ  $f(x)$ ; trong **khối phần dư (residual block)** chúng chỉ cần học phần dư  $g(x) = f(x) - x$  và đầu ra là

$$f(x) = g(x) + x.$$

Khi ánh xạ tối ưu gần với ánh xạ đồng nhất, mạng chỉ cần đẩy  $g$  về 0 - dễ học hơn nhiều; đường tắt (shortcut) cũng giúp gradient truyền thẳng về các lớp đầu, khắc phục triệt tiêu gradient. Khối phần dư cơ bản gồm Conv  $3 \times 3$  - BN - ReLU -

Conv  $3 \times 3$  - BN, cộng với đường tắt rồi qua ReLU; khi số kênh hoặc kích thước thay đổi, đường tắt dùng tích chập  $1 \times 1$  (kèm stride) để biến đổi đầu vào cho khớp. ResNet-18 xếp các khối này thành bốn giai đoạn (stage) 64 - 128 - 256 - 512 kênh và chiến thắng ImageNet 2015.

**DenseNet** (Huang et al., 2017) - trong mỗi khối dày đặc (dense block), mỗi lớp nhận bản đồ đặc trưng của **tất cả** các lớp trước (nối theo chiều kênh) và tạo thêm  $k$  kênh mới (growth rate), khuyến khích tái sử dụng đặc trưng; các khối được nối bởi lớp chuyển tiếp (BN, tích chập  $1 \times 1$ , gộp trung bình  $2 \times 2$ ) để giảm số kênh và kích thước.

**MobileNetV2** (Sandler et al., 2018) - kiến trúc nhẹ cho thiết bị di động, được dùng làm mạng nền trong Mô hình 3. Hai ý tưởng chính:

- *Tích chập tách biệt theo chiều sâu (depthwise separable convolution)*: tách một lớp tích chập  $k \times k$  thông thường thành tích chập theo chiều sâu (mỗi kênh một kernel  $k \times k$  riêng) và tích chập  $1 \times 1$  kết hợp các kênh; chi phí tính toán giảm khoảng  $k^2$  lần (gần 9 lần với  $k = 3$ ).
- *Khối phần dư đảo ngược với nút cổ chai tuyến tính (inverted residual, linear bottleneck)*:  $1 \times 1$  mở rộng số kênh (thường gấp 6) - depthwise  $3 \times 3$  -  $1 \times 1$  thu hẹp số kênh **không có hàm kích hoạt**; đường tắt nối các đầu “hẹp” khi stride bằng 1 và số kênh không đổi.

Mạng gồm một lớp tích chập đầu, 17 khối như trên và một lớp  $1 \times 1$  cuối 1.280 kênh; chỉ khoảng 3,5 triệu tham số (2,26 triệu khi bỏ phần phân loại) và khoảng 300 triệu phép nhân - cộng cho ảnh  $224 \times 224$ , được huấn luyện sẵn trên ImageNet (khoảng 1,28 triệu ảnh, 1.000 lớp; Deng et al., 2009).

### 1.7. Học chuyển giao (transfer learning) và tinh chỉnh (fine-tuning)

Học chuyển giao là quá trình áp dụng tri thức của một mô hình đã được huấn luyện trước (pretrained model) cho bài toán hiện tại. Các đặc trưng mà CNN học được trên tập dữ liệu lớn như ImageNet - đặc biệt ở các lớp đầu (cạnh, màu sắc, kết cấu) - mang tính tổng quát và có thể dùng lại cho nhiều bài toán khác (Yosinski et al., 2014). Lợi ích: hội tụ nhanh hơn, độ chính xác cao hơn, chi phí huấn luyện thấp hơn, hiệu quả khi dữ liệu ít.

Quy trình gồm ba pha:

1. **Huấn luyện trước (pretraining)**: mô hình được huấn luyện trên tập lớn (ImageNet) và học được đặc trưng tổng quát.
2. **Chuyển giao (transfer)**: lấy **mạng nền (base network)** - phân trích xuất đặc trưng sau khi bỏ các lớp kết nối đầy đủ ở đỉnh (`include_top=False`) - giữ nguyên trọng số đã huấn luyện, rồi gắn một MLP mới (khởi tạo ngẫu nhiên) để tính xác suất đầu ra. Có thể đóng băng toàn bộ mạng nền (trích xuất đặc trưng - feature extraction) và chỉ huấn luyện MLP.
3. **Tinh chỉnh (fine-tuning)**: mở băng một phần mạng nền (thường là các khối cuối, mang đặc trưng chuyên biệt của bài toán gốc) và huấn luyện tiếp trên dữ liệu mới với tốc độ học nhỏ.

Các lưu ý thực hành: (i) huấn luyện phần đầu ra mới trước rồi mới tinh chỉnh, tránh gradient lớn từ các lớp khởi tạo ngẫu nhiên phá hỏng trọng số đã học; (ii) dùng tốc độ học nhỏ ( $10^{-4}$  -  $10^{-5}$ ) khi tinh chỉnh; (iii) giữ các lớp batch normalization ở chế độ suy luận, vì thống kê của các mini-batch nhỏ trên tập dữ liệu mới kém ổn định; (iv) tiền xử lý ảnh **giống hệt** lúc huấn luyện trước - MobileNetV2 yêu cầu điểm ảnh trong khoảng  $[-1,1]$  ( $(x/127,5 - 1)$ ).

## 1.8. Tăng cường dữ liệu (data augmentation)

Hiệu năng kém của mô hình thường đến từ dữ liệu không đủ hoặc không đại diện, mất cân bằng lớp, kiến trúc quá phức tạp so với dữ liệu, hoặc khó khăn trong tối ưu. Tăng cường dữ liệu tạo thêm dữ liệu huấn luyện một cách nhân tạo bằng các phép biến đổi ngẫu nhiên giữ nguyên nhãn: xoay, lật, cắt; phóng to - thu nhỏ, dịch chuyển; điều chỉnh độ sáng, độ tương phản, màu sắc; thêm nhiễu. Mỗi epoch, mô hình nhìn thấy một phiên bản biến đổi khác của từng ảnh (“không bao giờ thấy cùng một ảnh hai lần”), nhờ đó giảm quá khớp, tăng khả năng tổng quát hoá và độ bền vững. Tăng cường dữ liệu chỉ áp dụng cho **tập huấn luyện**; tập kiểm định và tập kiểm tra giữ nguyên để đánh giá trung thực. Phép biến đổi phải giữ nguyên nhãn - ví dụ với bài toán độ tươi, không nên thay đổi sắc độ màu vì màu nâu, vết thâm là dấu hiệu của quả hỏng.

## 1.9. Học đa nhiệm (multi-task learning)

Học đa nhiệm (Caruana, 1997) huấn luyện một mô hình giải đồng thời nhiều bài toán liên quan: một **thân chung (shared trunk)** trích xuất đặc trưng, rẽ thành

nhiều **đầu ra (head)**, mỗi đầu cho một nhiệm vụ. Hàm mất mát tổng là tổng có trọng số các hàm mất mát thành phần

$$J = \sum_t \lambda_t J_t.$$

Với bài toán nhận diện loại quả và độ tươi:  $J = \lambda_1 \cdot \text{CE}(\text{loại quả}) + \lambda_2 \cdot \text{BCE}(\text{độ tươi})$ . Lợi ích: hai nhiệm vụ dùng chung đặc trưng cấp thấp (hình dạng, màu, kết cấu) nên cần ít tham số hơn hai mô hình riêng; nhiệm vụ này đóng vai trò chính quy hoá cho nhiệm vụ kia; và đầu ra có cấu trúc rõ ràng hơn so với gộp thành một bài toán 16 lớp. Trong Keras, mô hình nhiều đầu ra được xây dựng bằng **Functional API**, cho phép đồ thị tính toán rẽ nhánh mà API Sequential không biểu diễn được.

### 1.10. Đánh giá mô hình phân loại

Với ma trận nhầm lẫn (confusion matrix) của bài toán nhị phân gồm TP, FP, TN, FN:

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} \\ &= \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \end{aligned}$$

Precision cho biết trong các mẫu được dự đoán là dương tính có bao nhiêu phần đúng; recall cho biết mô hình tìm được bao nhiêu phần trong số các mẫu dương tính thực sự;  $F_1$  là trung bình điều hoà của hai đại lượng. Với bài toán nhiều lớp, các chỉ số được tính cho từng lớp rồi lấy trung bình không trọng số (**macro average**), để các lớp ít mẫu được coi trọng như lớp nhiều mẫu. Diện tích dưới đường cong ROC (ROC-AUC) đo khả năng xếp hạng: xác suất một mẫu dương tính ngẫu nhiên được cho điểm cao hơn một mẫu âm tính ngẫu nhiên. Ma trận nhầm lẫn cho thấy cụ thể lớp nào bị nhầm sang lớp nào.

### 1.11. Giải thích mô hình bằng Grad-CAM

CNN thường bị xem là “hộp đen”. Grad-CAM (Selvaraju et al., 2017) chỉ ra vùng ảnh nào ảnh hưởng mạnh nhất đến một quyết định. Gọi  $A^k$  là bản đồ đặc trưng thứ  $k$  của lớp tích chập cuối và  $y^c$  là điểm số của quyết định cần giải thích (ví dụ log-

odds của xác suất “hỏng”). Trọng số của từng kênh là gradient trung bình theo không gian

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k},$$

và bản đồ nhiệt (heatmap) là tổ hợp tuyến tính các bản đồ đặc trưng, giữ lại phần dương:

$$L_{\text{Grad-CAM}}^c = \text{ReLU} \left( \sum_k \alpha_k^c A^k \right).$$

Bản đồ nhiệt được phóng to về kích thước ảnh và phủ lên ảnh gốc: vùng sáng là vùng ủng hộ mạnh cho quyết định. Grad-CAM giúp kiểm tra mô hình có nhìn vào đúng vùng quả (vết thâm, nấm mốc) hay đang dựa vào các yếu tố không liên quan như nền ảnh.

## PHẦN 2. ỨNG DỤNG THỰC TIỄN

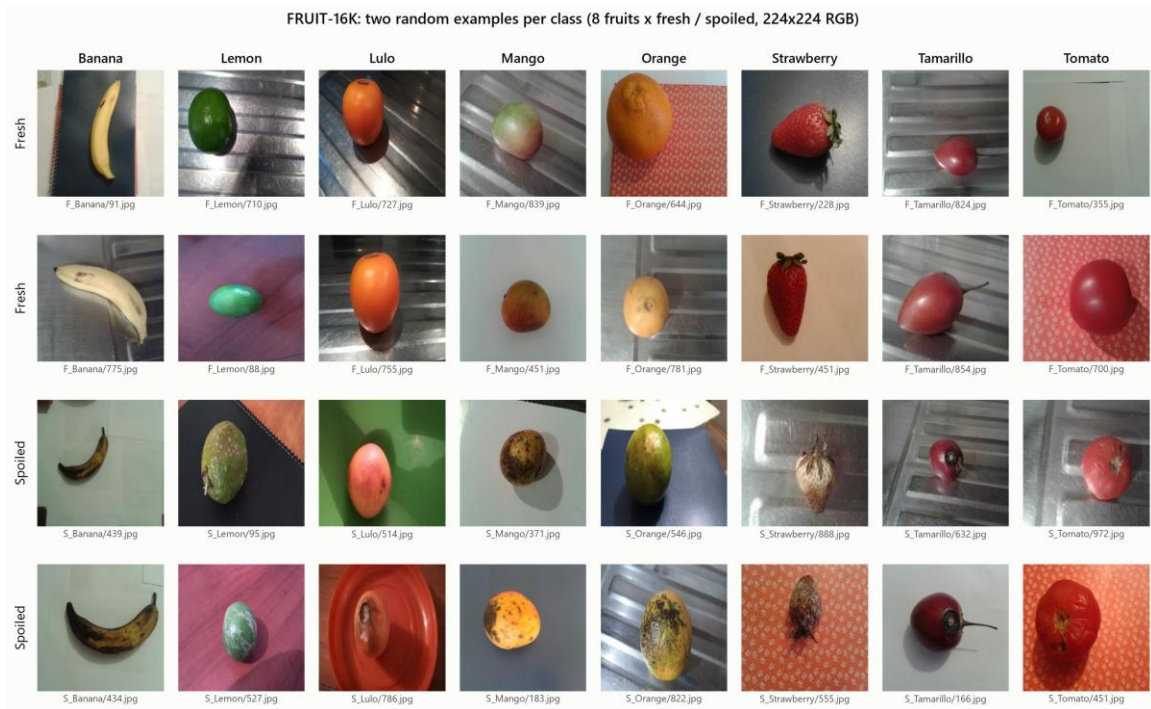
### I. Mô tả bài toán

- **Tên đề tài:** Phân loại trái cây tươi và hỏng bằng mạng nơ-ron tích chập (Fresh and Rotten Fruit Classification).
- **Bối cảnh:** kiểm tra chất lượng trái cây trước khi phân phối hiện vẫn chủ yếu làm thủ công, tốn công và dễ sai sót. Một hệ thống dựa trên ảnh có thể đồng thời nhận ra loại quả và đánh giá quả còn tươi hay đã hỏng sẽ hỗ trợ tự động hoá khâu phân loại và kiểm soát chất lượng - đúng mục đích mà nhóm tác giả bộ dữ liệu đặt ra.
- **Mục tiêu nghiên cứu:**
  1. Xây dựng và so sánh ba mô hình CNN với mức độ phức tạp tăng dần trên cùng một bộ dữ liệu: (1) CNN tuần tự đơn giản tự thiết kế; (2) CNN phức tạp tự thiết kế từ các khối phần dư, học đa nhiệm với hai đầu ra; (3) mô hình học chuyển giao từ MobileNetV2 huấn luyện sẵn trên ImageNet, có tinh chỉnh.
  2. Đánh giá ảnh hưởng của tăng cường dữ liệu và của số lớp được tinh chỉnh (nghiên cứu cắt bỏ - ablation).
  3. Dùng Grad-CAM để kiểm tra vùng ảnh nào quyết định dự đoán tươi / hỏng.
- **Đầu vào:** một ảnh màu RGB chụp một quả, kích thước  $224 \times 224$  điểm ảnh.
- **Đầu ra:**
  - loại quả - một trong 8 lớp: chuối (banana), chanh (lemon), lulo, xoài (mango), cam (orange), dâu tây (strawberry), cà chua thân gỗ (tamarillo), cà chua (tomato);
  - độ tươi - tươi (fresh) hoặc hỏng (spoiled), kèm xác suất  $P$ (hỏng).  
**Lớp “hỏng” được chọn là lớp dương tính** (positive class), vì mục tiêu kiểm soát chất lượng là phát hiện quả hỏng;
  - tương đương với một nhãn kết hợp trong 16 lớp (ví dụ Mango\_spoiled), mã hoá nhãn kết hợp  $= 2 \times$  nhãn loại quả + nhãn độ tươi.
- **Tóm tắt các công việc đã thực hiện:**

1. Kiểm định dữ liệu: đọc thử toàn bộ ảnh, thống kê kích thước - kênh màu, phát hiện ảnh trùng lặp (SHA-256) và ảnh gán nhãn mâu thuẫn.
2. Làm sạch dữ liệu và chia tập train / validation / test theo tỉ lệ 70 / 15 / 15, phân tầng theo lớp và **chống rò rỉ dữ liệu** (giữ các ảnh chụp liên tiếp gần giống nhau trong cùng một tập).
3. Xây dựng đường ống dữ liệu dùng chung: giải mã ảnh, chuẩn hoá, tăng cường dữ liệu chỉ cho tập huấn luyện.
4. Thiết kế, huấn luyện và tinh chỉnh ba mô hình với cùng một quy trình (dùng sớm, lưu mô hình tốt nhất, giảm tốc độ học).
5. Nghiên cứu cắt bỏ: có / không tăng cường dữ liệu; số khối MobileNetV2 được tinh chỉnh.
6. Đánh giá trên tập kiểm tra bằng accuracy, precision, recall, F1-score, ma trận nhầm lẫn và thời gian suy luận; trực quan hoá Grad-CAM và phân tích lỗi.

## II. Mô tả bộ dữ liệu

- **Tên và nguồn:** Spoiled and Fresh Fruit Inspection Dataset (FRUIT-16K) - Pachon Suescun, Pinzón Arenas và Jiménez-Moreno, Đại học Quân sự Nueva Granada (Colombia), công bố năm 2020 trên Mendeley Data.
- **Link tải:** <https://data.mendeley.com/datasets/6ps7gtp2wg/1> (DOI: 10.17632/6ps7gtp2wg.1), giấy phép CC BY 4.0.
- **Số lượng ảnh:** 16.000 ảnh JPEG, gồm 8 loại quả; mỗi loại 2.000 ảnh, một nửa là quả tươi và một nửa là quả hỏng. Dữ liệu được tổ chức thành 16 thư mục F\_<Quả> (tươi) và S\_<Quả> (hỏng), mỗi thư mục 1.000 ảnh đánh số 1.jpg - 1000.jpg theo thứ tự chụp.
- **Đối tượng trong ảnh:** mỗi ảnh chụp một quả thuộc một trong 8 loại trên, đặt trên nhiều nền khác nhau (bồn rửa inox, mặt bàn gỗ, bìa sổ, khăn trải bàn, giấy trắng), với khoảng cách chụp, góc xoay và ánh sáng thay đổi. Quả hỏng có các dấu hiệu như vết thâm, đốm đen, nấm mốc, vỏ nhăn, đổi màu (Hình 1).
- **Kích thước ảnh:** tất cả 16.000 ảnh đều có kích thước  $224 \times 224$  điểm ảnh, 3 kênh màu RGB, dung lượng trung bình khoảng 6 KB (nén JPEG mạnh). Giá trị trung bình / độ lệch chuẩn của điểm ảnh (thang [0, 1]) sau làm sạch: R 0,494 / 0,209; G 0,424 / 0,195; B 0,389 / 0,197.



Hình 1. Ví dụ ảnh của 16 lớp (8 loại quả x tươi / hỏng)

**Kết quả kiểm định dữ liệu.** Không có ảnh lỗi hay không đọc được; mọi ảnh đều  $224 \times 224$  RGB. Tuy nhiên phát hiện hai vấn đề quan trọng (Hình 2):

- **1.273 ảnh trùng lặp hoàn toàn** (trùng mã băm SHA-256), luôn là các cặp ảnh liền nhau (ví dụ 99.jpg = 100.jpg), tập trung ở 4 thư mục: F\_Lemon (445), F\_Lulo (442), F\_Tamarillo (318), S\_Tomato (68).
- **1 ảnh gán nhãn mâu thuẫn:** F\_Banana/1.jpg (nhãn “tươi”) chính là ảnh S\_Banana/6.jpg (nhãn “hỏng”) - khoảng cách perceptual hash bằng 0; quả chuối trong ảnh rõ ràng đã hỏng.

Ngoài ra, ảnh được chụp theo từng loạt nên các ảnh liền nhau thường gần như giống hệt (cùng quả, cùng tư thế, cùng nền).

**A. Exact duplicates (1,273 byte-identical copies in 4 folders)**



**B. Consecutive burst frames are near-copies**



**C. Label conflict between folders**



Same photo stored under both labels  
(pHash distance 0, thumbnail distance 0.0009).  
F\_Banana/1.jpg is excluded as mislabelled.

*Hình 2. Ảnh trùng lặp hoàn toàn, các ảnh liên tiếp gần giống nhau và ảnh gán nhãn mâu thuẫn*

### III. Thiết kế mô hình CNN

#### 1. Tiền xử lý dữ liệu: làm sạch, chuẩn hoá và tăng cường

**Làm sạch.** Loại 1.273 bản sao trùng lặp (giữ ảnh đầu tiên của mỗi cặp) và loại ảnh gán nhãn sai F\_Banana/1.jpg. Còn lại **14.726 ảnh**; các lớp trở nên hơi mất cân bằng: Lemon\_fresh 555, Lulo\_fresh 558, Tamarillo\_fresh 682, Tomato\_spoiled 932, Banana\_fresh 999, các lớp còn lại 1.000 ảnh.

**Chia tập chống rò rỉ dữ liệu.** Nếu chia ngẫu nhiên từng ảnh, các ảnh gần như giống hệt nhau của cùng một loạt chụp sẽ rơi vào cả tập huấn luyện và tập kiểm tra, khiến kết quả kiểm tra phản ánh khả năng “nhớ” hơn là khả năng tổng quát

hoá. Chúng tôi đo, với mỗi ảnh kiểm tra, ảnh huấn luyện cùng lớp giống nó nhất theo perceptual hash (pHash, 64 bit) và so sánh ba cách chia (Bảng 1).

*Bảng 1. Tỷ lệ ảnh kiểm tra có “bản sao gần giống” trong tập huấn luyện*

| Cách chia                                                             | pHash $\leq 6$ | pHash $\leq 10$ |
|-----------------------------------------------------------------------|----------------|-----------------|
| Chia ngẫu nhiên phân tầng (từng ảnh)                                  | 9,7 %          | 32,9 %          |
| Giữ 25 ảnh liên tiếp trong cùng một tập                               | 3,9 %          | 20,3 %          |
| <b>Cách chia được chọn: nhóm 25 ảnh liên tiếp + gộp ảnh gần trùng</b> | <b>1,2 %</b>   | <b>14,6 %</b>   |

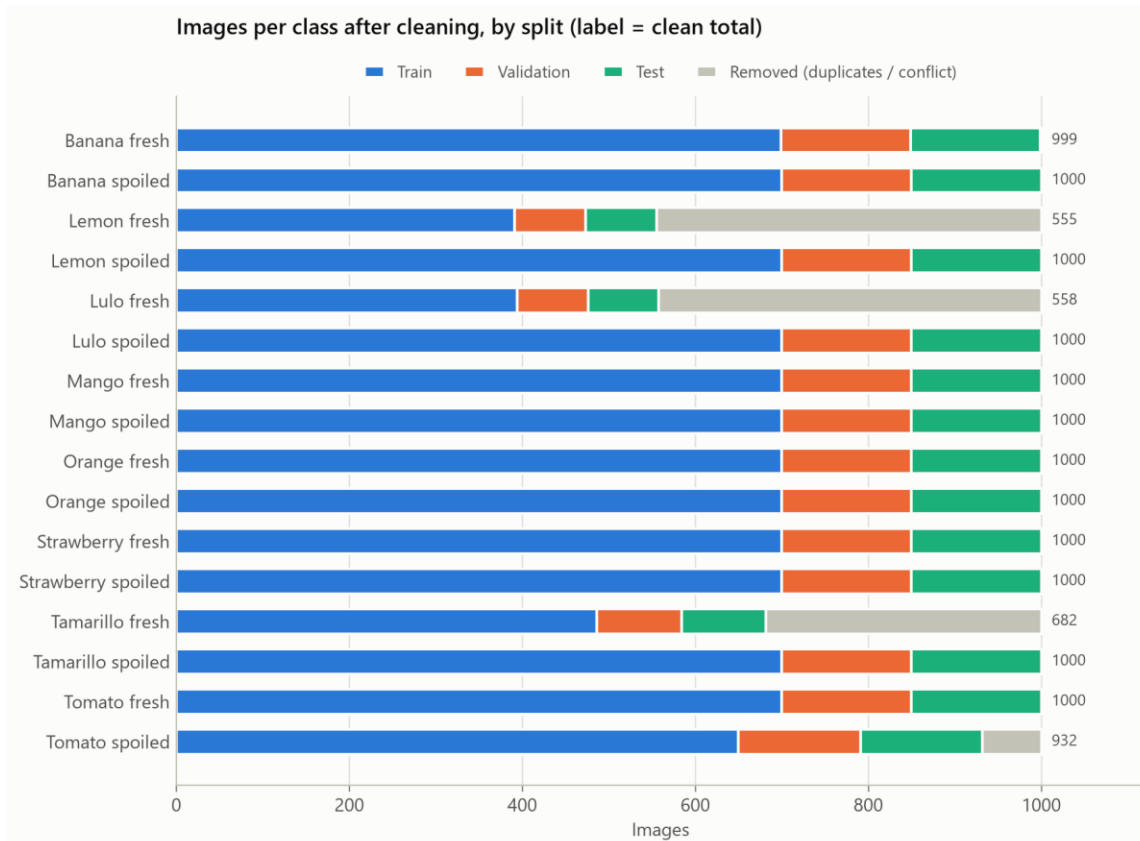
Cách chia được chọn gồm hai bước:

1. **Lập nhóm:** trong mỗi thư mục lớp, cứ 25 ảnh liên tiếp tạo thành một nhóm (một loạt chụp); hai ảnh cùng lớp cũng được gộp vào một nhóm nếu chúng gần trùng nhau - pHash cách nhau không quá 6 bit **và** hoặc điểm ảnh gần như giống hệt (sai khác trung bình trên ảnh thu nhỏ  $16 \times 16$  không quá 0,03), hoặc cách nhau không quá 50 ảnh (cùng một quả chụp lại dưới ánh sáng khác).
2. **Phân nhóm vào các tập:** toàn bộ nhóm được đưa vào train, validation hoặc test, thực hiện **riêng cho từng lớp trong 16 lớp** (phân tầng), nhóm lớn xếp trước, mỗi nhóm vào tập còn thiếu nhiều nhất so với tỷ lệ 70 / 15 / 15 (seed 42, kết quả luôn tái lập được).

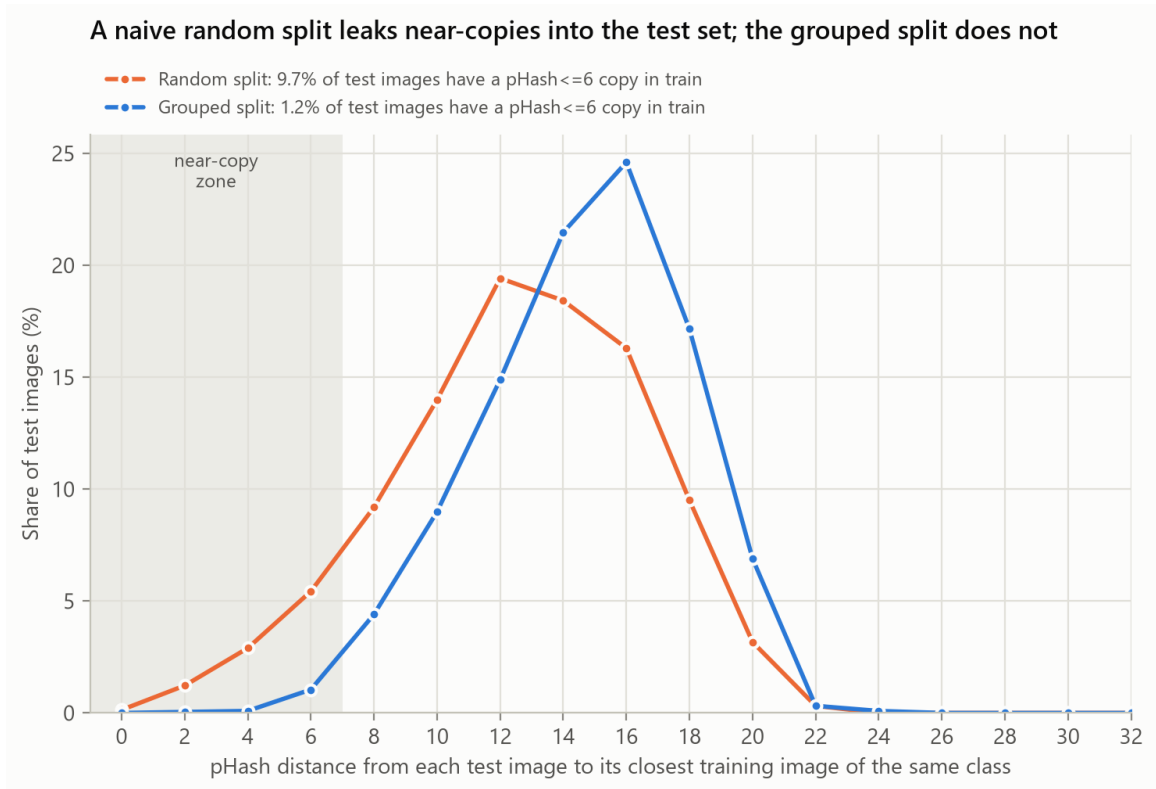
Kiểm tra sau khi chia: không có ảnh, mã băm hay nhóm nào xuất hiện ở hai tập; không có cặp ảnh gần trùng nào nằm ở hai tập khác nhau; tỷ lệ mỗi lớp lệch khỏi mục tiêu không quá 1,3 điểm phần trăm (Hình 3, Hình 4).

*Bảng 2. Số mẫu trong ba tập dữ liệu*

| Tập                    | Số ảnh | Tỷ lệ  | Mục đích sử dụng                                                                               |
|------------------------|--------|--------|------------------------------------------------------------------------------------------------|
| Huấn luyện (training)  | 10.320 | 70,1 % | học trọng số mô hình                                                                           |
| Kiểm định (validation) | 2.203  | 15,0 % | dừng sớm, chọn epoch tốt nhất, điều chỉnh tốc độ học, quyết định trong các thí nghiệm ablation |
| Kiểm tra (test)        | 2.203  | 15,0 % | chỉ dùng một lần để đánh giá cuối cùng                                                         |



Hình 3. Số ảnh mỗi lớp sau làm sạch, theo tập



Hình 4. Khoảng cách pHash từ mỗi ảnh kiểm tra đến ảnh huấn luyện giống nhất: chia ngẫu nhiên và chia theo nhóm

**Thay đổi kích thước và chuẩn hoá.** Ảnh được giải mã thành tensor  $224 \times 224 \times 3$  (đúng kích thước gốc nên không cần thay đổi kích thước). Bước chuẩn hoá được đặt thành **lớp đầu tiên của mỗi mô hình**, nhờ vậy mô hình đã lưu có thể nhận trực tiếp ảnh gốc khi triển khai:

- Mô hình 1 và 2: Rescaling( $1/255$ ) đưa điểm ảnh về  $[0,1]$ ;
- Mô hình 3:  $x/127,5 - 1$  đưa về  $[-1,1]$  - đúng miền giá trị mà trọng số ImageNet của MobileNetV2 được huấn luyện.

**Tăng cường dữ liệu (chỉ cho tập huấn luyện).** Dùng các lớp tiền xử lý của Keras, sinh giá trị ngẫu nhiên mới ở mỗi epoch: lật ngang và lật dọc, xoay tối đa  $\pm 36^\circ$ , phóng to - thu nhỏ  $\pm 15\%$ , độ sáng  $\pm 15\%$ , độ tương phản  $\pm 15\%$  (Hình 5). Không thay đổi sắc độ màu vì màu sắc (nâu hoá, đốm thâm, nấm mốc) là dấu hiệu chính của quả hỏng. Đường ống tf.data lưu đệm (cache) dữ liệu ảnh đã nén trong bộ nhớ, xáo trộn tập huấn luyện mỗi epoch, chia batch 32 và nạp trước (prefetch).

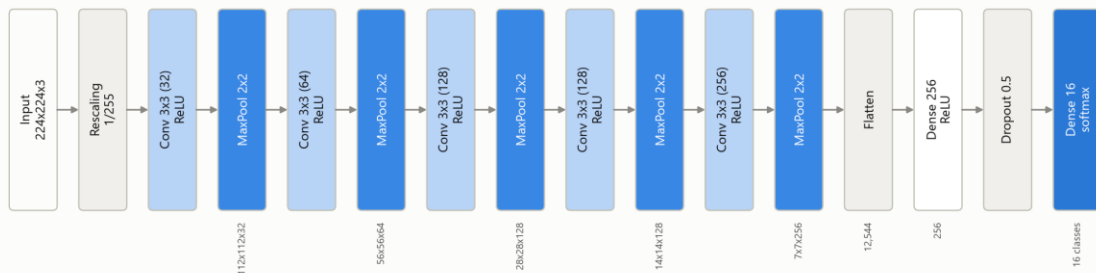


Hình 5. Một ảnh huấn luyện qua tám lần tăng cường dữ liệu ngẫu nhiên

## 2. Mô hình 1 - CNN tuần tự đơn giản

Mô hình 1 được xây dựng bằng **Keras Sequential API** theo mẫu kinh điển của LeNet / AlexNet: năm khối [Tích chập  $3 \times 3$  - ReLU - Gộp cực đại  $2 \times 2$ ] với số kênh 32 - 64 - 128 - 128 - 256, sau đó đuôi phẳng và hai lớp kết nối đầy đủ (Hình 6, Bảng 3). Lớp đầu ra softmax dự đoán trực tiếp **16 lớp kết hợp**; loại quả và độ tươi được suy ra từ 16 xác suất:  $P(\text{quả}) = P(\text{quả, tươi}) + P(\text{quả, hỏng})$  và  $P(\text{hỏng})$  bằng tổng 8 xác suất của các lớp “hỏng”. Hàm mất mát: sparse categorical cross-entropy.

Model 1 - simple sequential CNN (3.75 M parameters): 5 x [Conv-ReLU-MaxPool] + fully connected layers



Hình 6. Kiến trúc Mô hình 1

Bảng 3. Các lớp của Mô hình 1

| Lớp                               | Kích thước đầu ra | Số tham số |
|-----------------------------------|-------------------|------------|
| Đầu vào + Rescaling(1/255)        | 224 x 224 x 3     | 0          |
| Conv 3x3, 32, ReLU + MaxPool 2x2  | 112 x 112 x 32    | 896        |
| Conv 3x3, 64, ReLU + MaxPool 2x2  | 56 x 56 x 64      | 18.496     |
| Conv 3x3, 128, ReLU + MaxPool 2x2 | 28 x 28 x 128     | 73.856     |
| Conv 3x3, 128, ReLU + MaxPool 2x2 | 14 x 14 x 128     | 147.584    |
| Conv 3x3, 256, ReLU + MaxPool 2x2 | 7 x 7 x 256       | 295.168    |
| Flatten                           | 12.544            | 0          |

| Lớp                           | Kích thước đầu ra | Số tham số       |
|-------------------------------|-------------------|------------------|
| Dense 256, ReLU + Dropout 0,5 | 256               | 3.211.520        |
| Dense 16, softmax             | 16                | 4.112            |
| <b>Tổng</b>                   |                   | <b>3.751.632</b> |

Phần lớn tham số (86 %) nằm ở lớp Dense đầu tiên sau Flatten - đặc điểm điển hình của kiến trúc “Flatten + Dense” mà các mạng hiện đại thay bằng gộp trung bình toàn cục.

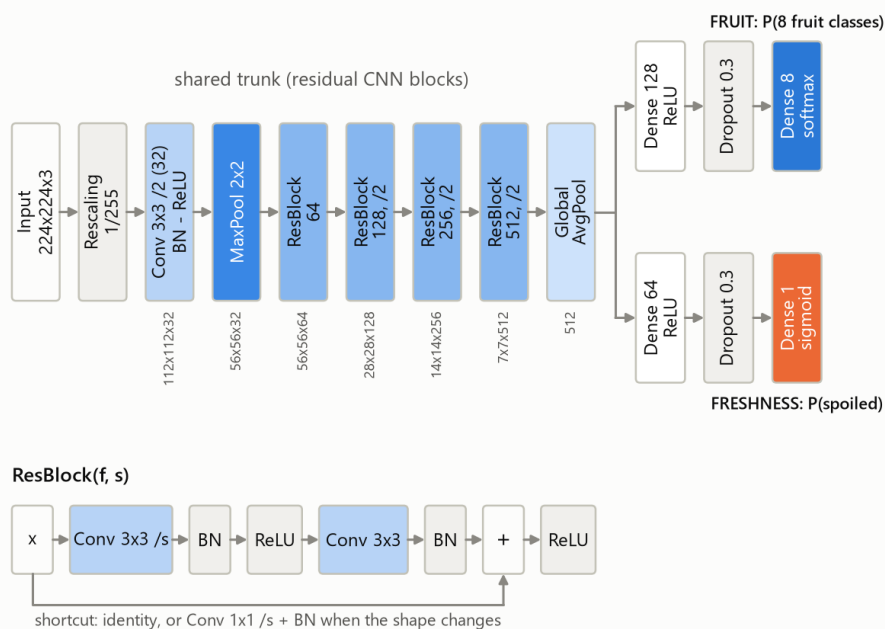
### 3. Mô hình 2 - CNN phức tạp đa nhiệm (Functional API)

Mô hình 2 được xây dựng bằng **Keras Functional API**. Thân mạng chung được ghép từ các **khối phần dư (residual block)** kiểu ResNet có chuẩn hoá theo lô (mỗi giai đoạn một khối, tương tự ResNet-10), sau đó rẽ thành **hai nhánh đầu ra song song** - học đa nhiệm (Hình 7):

- phần đầu (stem): Conv  $3 \times 3$  stride 2 (32 kênh) - BN - ReLU - MaxPool  $2 \times 2 \rightarrow 56 \times 56 \times 32$ ;
- bốn giai đoạn, mỗi giai đoạn một khối phần dư: 64 kênh ( $56 \times 56$ ), 128 kênh stride 2 ( $28 \times 28$ ), 256 kênh stride 2 ( $14 \times 14$ ), 512 kênh stride 2 ( $7 \times 7$ );
- khối phần dư ResBlock( $f, s$ ): Conv  $3 \times 3$  stride  $s$  - BN - ReLU - Conv  $3 \times 3$  - BN, cộng với đường tắt rồi qua ReLU; đường tắt là ánh xạ đồng nhất, hoặc Conv  $1 \times 1$  stride  $s$  + BN khi kích thước thay đổi;
- gộp trung bình toàn cục  $\rightarrow$  vector 512 chiều, rồi rẽ hai nhánh:
  - **nhánh loại quả:** Dense 128 - ReLU - Dropout 0,3 - Dense 8 softmax;
  - **nhánh độ tươi:** Dense 64 - ReLU - Dropout 0,3 - Dense 1 sigmoid =  $P(\text{hồng})$ .

Hàm mất mát tổng:  $J = 1,0 \cdot \text{CE}_{\text{loại quả}} + 1,0 \cdot \text{BCE}_{\text{độ tươi}}$ . Tổng số tham số: 4.986.345 (trong đó 5.824 tham số không huấn luyện là trung bình và phương sai trượt của BN). So với Mô hình 1, Mô hình 2 sâu hơn (9 lớp tích chập trên nhánh chính, cộng 4 tích chập  $1 \times 1$  ở các đường tắt), dùng BN và kết nối tắt để huấn luyện ổn định, dùng GAP thay cho Flatten + Dense, và tách bài toán thành hai nhiệm vụ có ý nghĩa thay vì 16 lớp rời rạc.

Model 2 - multi-task CNN (Functional API, 4.99 M parameters): residual trunk shared by two heads



Hình 7. Kiến trúc Mô hình 2 và cấu trúc khối phân dư

#### 4. Mô hình 3 - Học chuyển giao và tinh chỉnh MobileNetV2

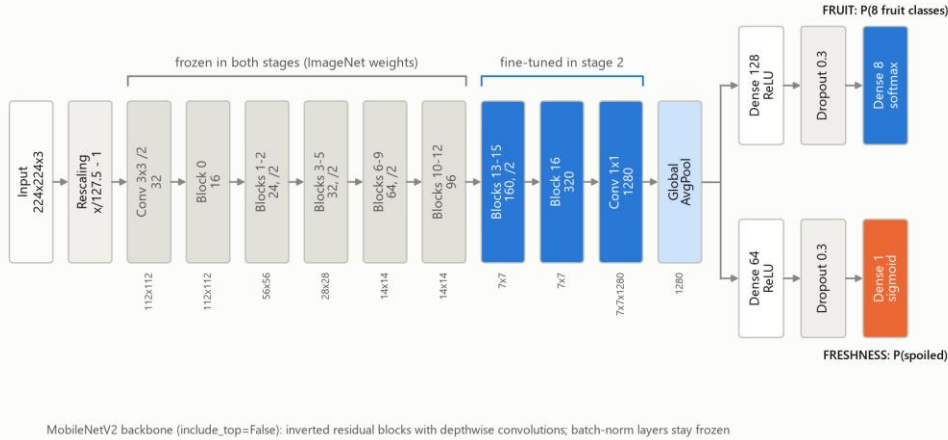
Mô hình 3 dùng **MobileNetV2** huấn luyện sẵn trên **ImageNet** làm mạng nền (include\_top=False, bỏ phần phân loại 1.000 lớp), lớp tiền xử lý  $x/127,5 - 1$ , và **gắn đúng hai nhánh đầu ra của Mô hình 2** (dùng chung một hàm xây dựng), để so sánh công bằng giữa mạng tự thiết kế và mạng chuyển giao (Hình 8). Tổng số tham số: 2.505.033 - mô hình nhỏ nhất trong ba mô hình nhưng mang tri thức từ 1,28 triệu ảnh ImageNet.

Huấn luyện gồm hai giai đoạn:

- **Giai đoạn 1 - trích xuất đặc trưng:** đóng băng toàn bộ mạng nền (2.257.984 tham số không huấn luyện), chỉ huấn luyện hai nhánh đầu ra (247.049 tham số); Adam, tốc độ học  $10^{-3}$ , tối đa 12 epoch.
- **Giai đoạn 2 - tinh chỉnh:** các khối đầu vẫn đóng băng; mở băng mạng nền từ lớp block\_13\_expand trở đi (các khối 13 - 16 và lớp tích chập  $1 \times 1$  cuối - 38 lớp, 1.910.409 tham số huấn luyện được), tốc độ học nhỏ  $10^{-4}$ , tối đa

25 epoch. Các lớp batch normalization của mạng nền luôn ở chế độ suy luận.

Model 3 - transfer learning: MobileNetV2 (ImageNet) + the same two heads; stage 1 trains the heads, stage 2 also fine-tunes blocks 13-16



Hình 8. Kiến trúc Mô hình 3: phần đóng băng và phần được tinh chỉnh

## 5. Quy trình huấn luyện chung

Cả ba mô hình dùng cùng tập dữ liệu, cùng batch 32, bộ tối ưu Adam và cùng các callback: lưu mô hình có val\_loss thấp nhất (ModelCheckpoint), dừng sớm sau 8 epoch không cải thiện và khôi phục trọng số tốt nhất (EarlyStopping), giảm tốc độ học 0,3 lần sau 3 epoch không cải thiện (ReduceLROnPlateau, tối thiểu  $10^{-6}$ ). Mô hình 1 và 2 huấn luyện tối đa 40 epoch với tốc độ học  $10^{-3}$ . Tập kiểm tra không được dùng trong bất kỳ quyết định nào trong quá trình huấn luyện. Toàn bộ thí nghiệm chạy trên CPU Intel Core i9-13900H (TensorFlow 2.21, Keras 3.15).

### Các thí nghiệm cắt bỏ (ablation):

- *Tăng cường dữ liệu*: mỗi mô hình được huấn luyện lại không có tăng cường dữ liệu.
- *Số lớp được tinh chỉnh (Mô hình 3)*: giai đoạn 2 được chạy lại từ **cùng một** mô hình giai đoạn 1, mở băng MobileNetV2 từ các vị trí khác nhau: không mở (chỉ trích xuất đặc trưng), từ khối 16, khối 13 (mô hình chính), khối 10, khối 6 và toàn bộ mạng.

## TÀI LIỆU THAM KHẢO

1. Caruana, R. (1997). Multitask learning. *Machine Learning*, 28(1), 41-75.
2. Chollet, F. (2017). *Deep learning with Python*. Manning Publications.
3. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 248-255).
4. Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics* (pp. 249-256).
5. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
6. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770-778).
7. Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4700-4708).
8. Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning* (pp. 448-456).
9. Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations*.
10. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems 25* (pp. 1097-1105).
11. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
12. Lin, M., Chen, Q., & Yan, S. (2013). Network in network. *arXiv preprint arXiv:1312.4400*.
13. Nguyễn, T. K. N. (2026). *Bài giảng Học sâu (Deep Learning)* [Slide bài giảng]. Trường Đại học Kinh tế Quốc dân.

14. Pachon Suescun, C. G., Pinzón Arenas, J. O., & Jiménez-Moreno, R. (2020). *Spoiled and fresh fruit inspection dataset* (Version 1) [Data set]. Mendeley Data. <https://doi.org/10.17632/6ps7gtp2wg.1>
15. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4510-4520).
16. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision* (pp. 618-626).
17. Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*.
18. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929-1958.
19. Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2015). Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 1-9).
20. Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems 27* (pp. 3320-3328).
21. Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into deep learning*. Cambridge University Press. (Bản dịch tiếng Việt: *Đắm mình vào học sâu*, nhóm dịch Vũ Hữu Tiệp và cộng sự.)
22. Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation* (pp. 265-283).
23. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.